Unit 1 Programming

1.1 Variables, constants, data types and casting

Objectives

By the end of the lesson, you should be able to answer:

Part 1 – How can variables and constants store data in a program?

- Variables, constants and identifiers
- Assignment and changing values

Part 2 – How are data types selected and converted?

- Integer, real, Boolean, character and string
- Casting between data types

Starter – What must the game remember?

A quiz game must remember:

- player's name
- current score
- whether the game has finished
- maximum possible score

Decide:

1. Which values might change while the program runs?
2. Which value should remain fixed?
3. What kind of data would each value contain?

Starter – Suggested answers

Answers

playerName changes between players and stores text.

score changes during the game and stores a whole number.

gameFinished switches between True and False.

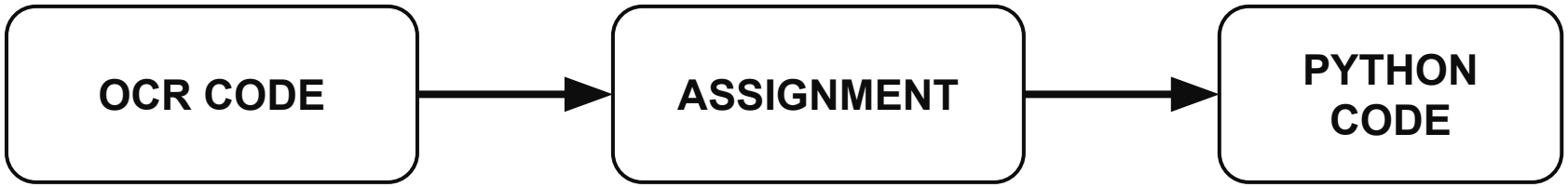
MAX_SCORE remains fixed while the program runs.

Today we will learn how programs label, store and control these values.

Reading code in OCR questions

Python and **OCR Exam Reference Language** may use different syntax, but the same programming idea can be valid in both.

For example, `score = 0` **assigns** the value 0 to the **identifier** score. Focus on what the instruction does.



Part 1

How can variables and constants store data in a program?

What is a variable?

A **variable** is a **named storage location** in a program.

It stores one value at a time. The **value can change** while the program runs, but the name stays the same.

score = 0

score is the **variable name**; **0** is the value currently stored.

Think: In a game, what would typical change the value of **score**?

Variable dissection

1) score is the variable name / identifier.
It identifies the storage location.

2) = is the **assignment operator**.
It stores the value on the right in score.

```
score = 0
```

3) 0 is the starting value.
It can be replaced while the program runs.

Why does a game need variables?

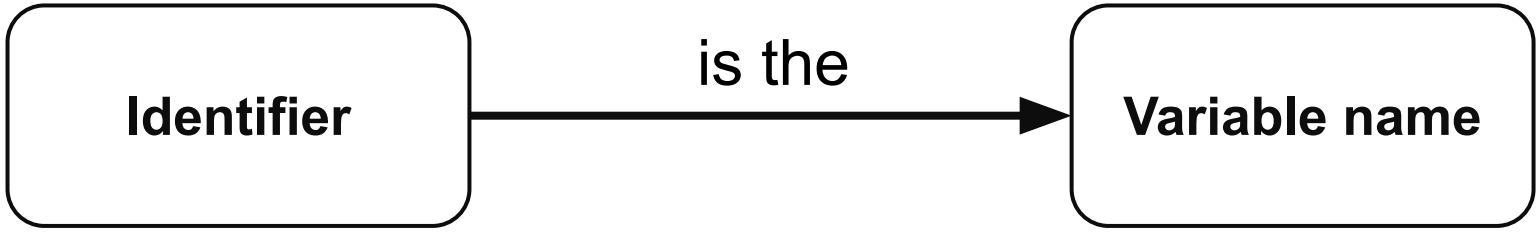
In a football game such as **EA Sports FC**, the **score** changes whenever a goal is scored. The program must remember the updated value so the scoreboard stays correct.

A **variable** is a named storage location whose value can change.



A variable is a labelled storage location

The **identifier** is the name of a variable.



When the value of a variable changes, the identifier stays the same, just like replacing the contents of a labelled box.

Identifier naming rules

Use meaningful names that describe the value being stored. This makes the code easier to understand.

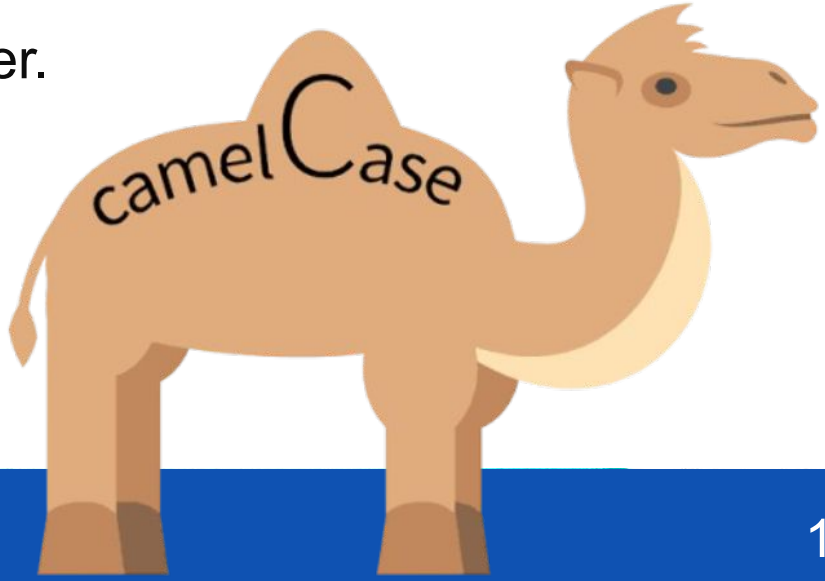
Example	Meaningful?
pn	**No**
playerName	**Yes**
ms	**No**
maximumScore	**Yes**
s	**No**
speed	**Yes**

Identifier naming rules

When choosing a variable name that contain multiple words, use a **camelCase** format.

- playerName
- maxScore
- enemyPosition

This makes reading the words easier.



Identifier naming rules

Which names are valid

Example	Valid?
first player	
firstPlayer	
1stPlayer	
player1	
_player1	
first_player	

Identifier naming rules

Identifiers cannot contain spaces or begin with a number.

Example	Valid?
first player	No
firstPlayer	Yes
1stPlayer	No
player1	Yes
_player1	Yes
first_player	Yes

Up next

We will model how a game stores and updates its score.

Model step 1 – Assign a starting value

score = 0

The **assignment operator** stores the value on the right in the variable named on the left.

Read it as: “score is assigned the value zero.”



Model step 2 – Calculate the new value

score = score + 10

Evaluate the **right-hand side** first. The expression uses the old value before the result is assigned.



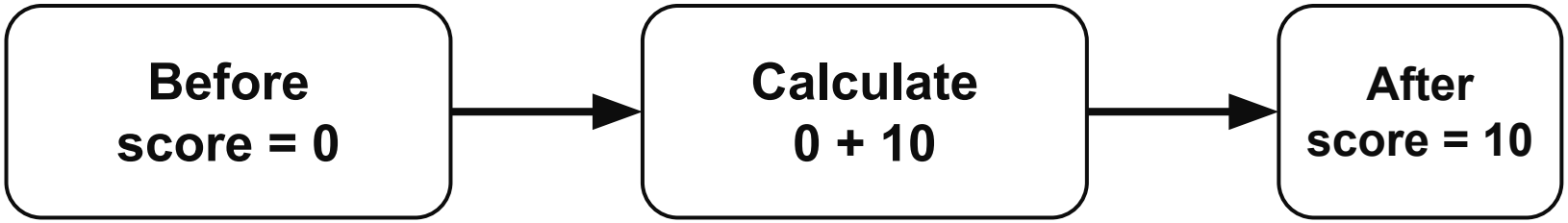
Model step 3 – Store the result

```
score = score + 10
```

The **calculated result** replaces the previous value.

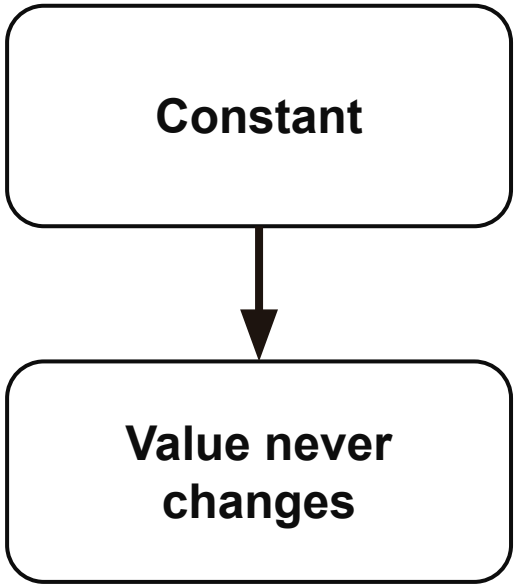
The identifier stays the same; only the stored value changes.

The score should change. Should every value be allowed to change?



What is a constant?

A **constant** is a named value that **cannot change** while the program runs.



Constant dissection

1) MAX_SCORE is the constant name / identifier. It identifies the storage location.

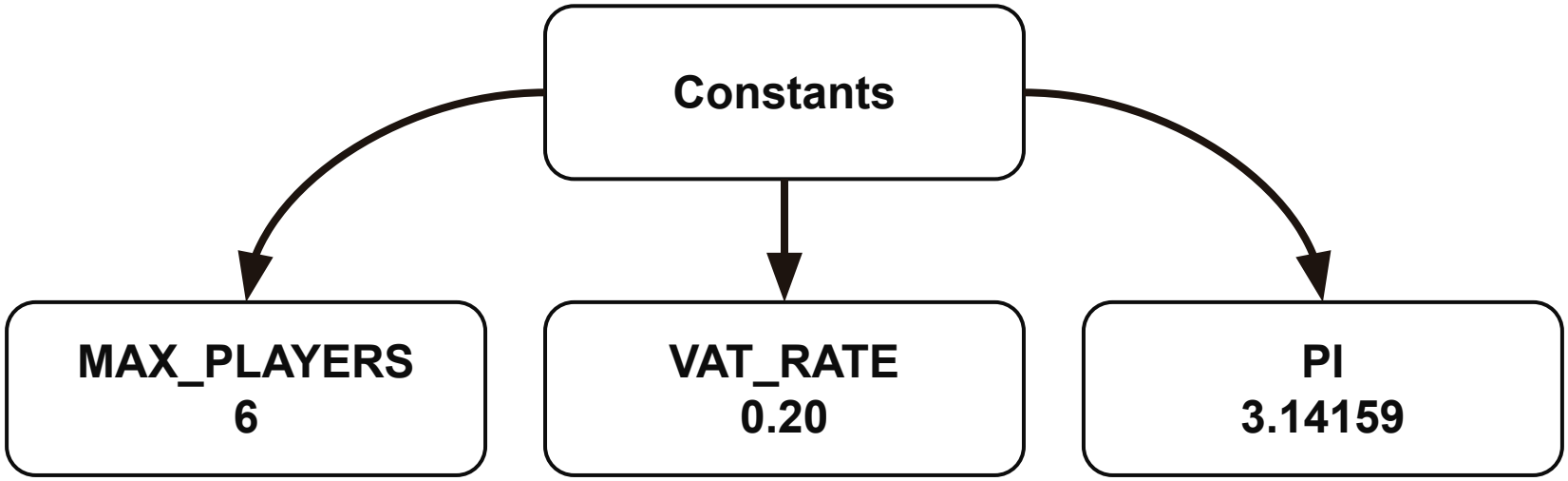
2) = is the **assignment operator**. It stores the value on the right in score.

```
MAX_SCORE = 100
```

3) 100 is the set value. It can NOT be replaced while the program runs.

Examples of constants

These values should remain fixed while their programs run:



- **Do you know why?**

Examples of constants

Answers

- **MAX_PLAYERS –** the game is designed for six players. Keeping the limit fixed means the same rule applies throughout the game.
- **VAT_RATE –** every transaction must use the same official tax rate until the rate is formally changed.
- **PI –** its value does not change because circle formulas rely on the same mathematical relationship.

Variable or constant?

Should the program ever assign a different value after it starts?

Use this question to decide whether each identifier should be a variable or a constant.

- **score**
- **playerName**
- **MAX_LIVES**

Variable or constant?

Answers

- **Yes — use a variable.** Its value is expected to change while the program runs.
- **No — use a constant.** Its value should remain fixed while the program runs.
- **score** and **playerName** are variables because their values may change.
- **MAX_LIVES** is a constant because the maximum player lives in a game should remain fixed.

Guided practice – Make the design decisions

A racing game stores the **driver's name**, **current lap**, whether the **race is finished** and the **fixed number of laps**.

For each item:

1. Choose a meaningful identifier.
2. Decide whether it is a variable or constant.
3. State its starting value.
4. Explain one choice to a partner.

Practical task – Build a simple scoreboard

Create a short program using your class language or OCR Reference Language.

Your program must:

1. store a player name and a starting score
2. define the fixed maximum as MAX_SCORE (the Python convention for a constant)
3. display the starting score
4. add 10 and display the updated score

Test it, then explain each assignment to a partner.

Multiple choice question

OCR Exam Reference Language

Which declaration creates a constant whose value should not change while the program runs?

A. score = score + 1

B. const MAX_PLAYERS = 6

C. playerName = input("Enter name: ")

D. found = False

Multiple choice question

Answers

Correct answer: B

A. score = score + 1

B. const MAX_PLAYERS = 6

C. playerName = input("Enter name: ")

D. found = False

Why:
In OCR Exam Reference Language, const declares a value that cannot change while the program runs. Uppercase naming also makes the constant easy to recognise.

Exam Question

A game stores a player's name, current score and maximum number of players.

Explain how variables, constants and data types should be used to store this information. (6 marks)

Exam Question

Answers

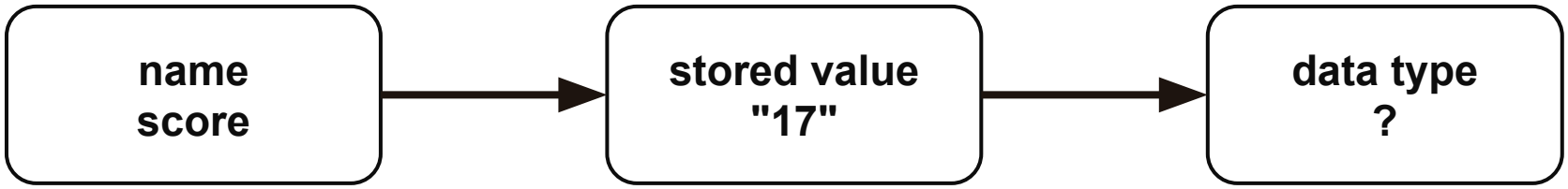
Answer guidance

- playerName stores text, so use a string variable. (1)
- score stores a whole number, so use an integer variable. (1)
- Variables can change while the program runs. (1)
- MAX_PLAYERS should be an integer constant. (1)
- Constants cannot change while the program runs. (1)
- Data types determine valid values and operations. (1)

Part 1 – Answer and bridge

Variables remember values that may change. **Constants** protect values that must stay fixed. Assignment stores a calculated result.

Next question: should "17" be treated as text or as a number?



Analyse the code

1. num1 = 10
2. num2 = 5
3. total = num1 + num2
4. difference = num1 - num2
5. product = num1 * num2
6. division = num1 / num2
7. print(total)
8. print(difference)
9. print(product)
10. print(division)

Complete the trace table

Line number	num1	num2	total	difference	product	division	Output
1	10	–	–	–	–	–	–
2							
3							
4							
5							
6							
7							
8							
9							
10							

Complete the trace table

Line number	num1	num2	total	difference	product	division	Output
1	10	–	–	–	–	–	–
2	–	5	–	–	–	–	–
3	–	–	15	–	–	–	–
4	–	–	–	5	–	–	–
5	–	–	–	–	50	–	–
6	–	–	–	–	–	2.0	–
7	–	–	–	–	–	–	15
8	–	–	–	–	–	–	5
9	–	–	–	–	–	–	50
10	–	–	–	–	–	–	2.0

Part 2

How are data types selected and converted?

Why does the type of a value matter?

A **data type** sets the rules for storing and using a value.

The type determines which **values and operations** are valid.

"17"
STRING
text: join or display

≠

17
INTEGER
number: add or compare

The five data types you must recognise

In OCR questions, recognise each type by the values it can store. Python represents a character using a one-character string.

INTEGER
whole number
27

REAL / FLOAT
fractional number
1.5

BOOLEAN
True or False

CHARACTER
one character
"A"

STRING
sequence of characters
"Max"

Choose a type by considering its purpose

Value	Suitable type	Why
ageInYears	Integer	Stores a whole number of years.
heightMetres	Real / Float	May include a fractional part, such as 1.72.
isLoggedIn	Boolean	Has only two states: True or False.
initial	Character	Stores one character.
phoneNumber	String	Not used in arithmetic and may begin with 0.

Guided practice – Select and justify

Choose the **most suitable data type** for each value:

1. numberOfLives
2. averageTemperature
3. hasPermission
4. menuChoice
5. postcode

Do not only name the type. **Justify** it by referring to the possible values or operations.

Guided practice – Select and justify

Answers

Choose the **most suitable data type** for each value:

1. numberOfLives **(Integer)**
2. averageTemperature **(Float)**
3. hasPermission **(Boolean)**
4. menuChoice **(Char or Integer)**
5. Postcode **(String)**

Do not only name the type. **Justify** it by referring to the possible values or operations.

When input arrives as text

A cinema app asks, “How many tickets?”

The user types 3, but **input** is received as the **string "3"**. The program needs a **number** before it can calculate 3 × £8.

Think: what must change—the value, its type, or both?

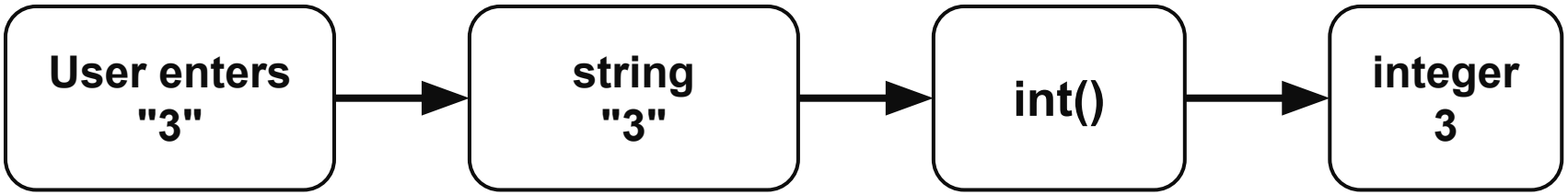
Next: we will see how casting solves this problem using syntax shared by Python and OCR Exam Reference Language.

Casting temporarily changes a data type

Casting converts a value from one data type to another when the program needs to use it differently.

A common example occurs with **user input**: the input is received as a **string**, even when the user types digits.

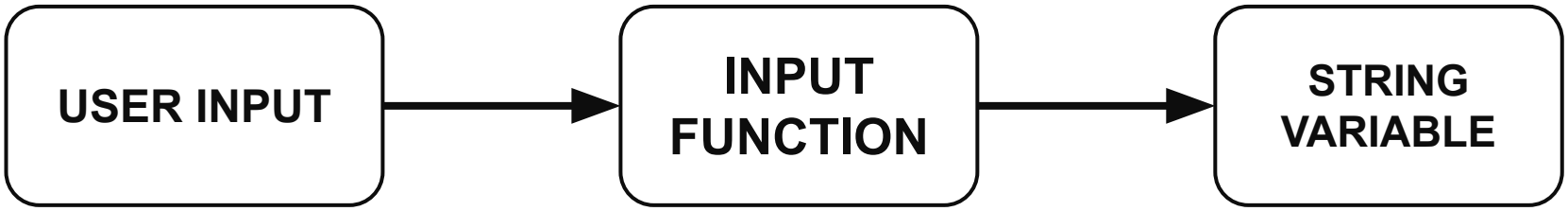
Input `"3"` → cast with `int()` → integer `3` → use in arithmetic



Model step 1 – Receive the input as text

ticketsText = input("How many tickets? ")

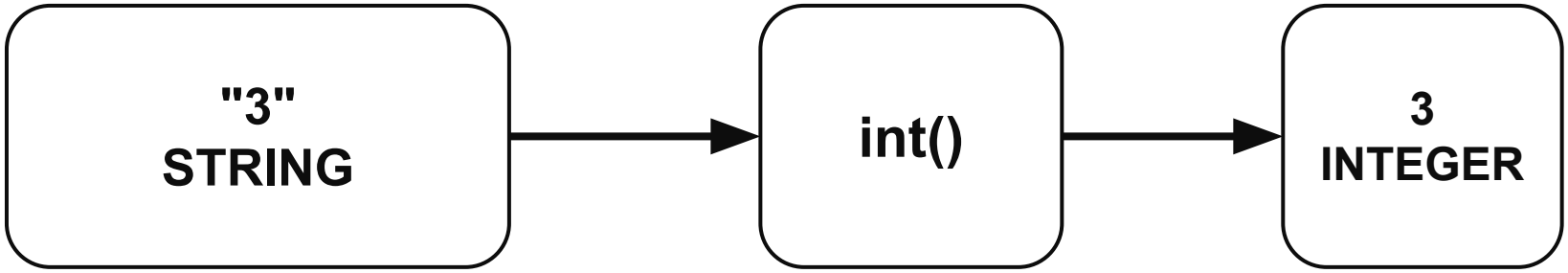
Suppose the user enters 3. Input returns text, so ticketsText stores "3" as a **string**.



Model step 2 – Cast the string

tickets = int(ticketsText)

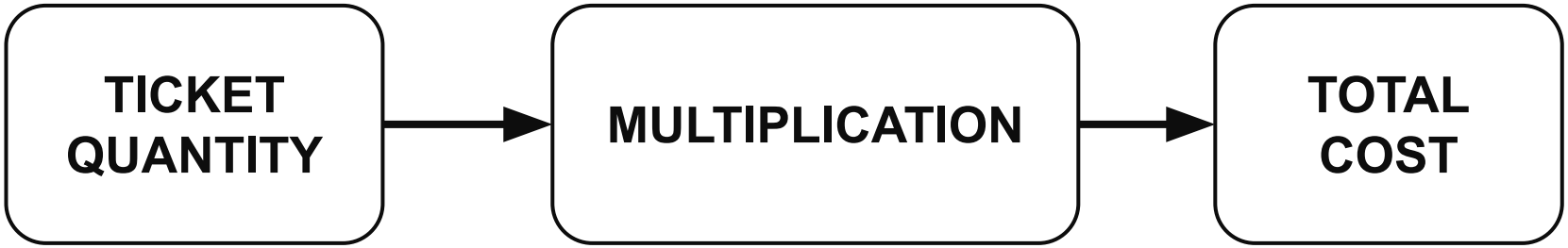
int() converts the numeric string "3" into the integer 3.



Model step 3 – Use the converted value

totalCost = tickets * 8

Because tickets stores an **integer**, the program can multiply it by the ticket price.



Can every value be cast?

Python code

Predict whether each conversion will succeed or cause an error. Be ready to explain why.

```
int("25")
```

```
int("A")
```

```
float("25.0")
```

```
str(25)
```

Casting check – Answers

Answers

- **int("25")** succeeds: the text contains digits that represent a whole number, so the result is integer 25.
- **int("A")** causes an error: the text does not represent a whole number.
- **float("25.0")** succeeds: the text represents a decimal number, so the result is real 25.0.
- **str(25)** succeeds: integer 25 is converted to the string "25".

Casting works only when the value can be represented by the target type.

Practical task – Ticket calculator

Write a program that:

1. asks for the number of tickets
2. casts the input to an integer
3. calculates the total at £8 per ticket
4. displays the result

Before running it:

- predict what is stored after each line
- test 3, 0 and another valid value

Extension: What happens if the user enters three as a word?

Multiple choice question

Python / OCR Exam Reference Language

Which statement correctly converts the text value "25" into an integer?

A. age = str("25")

B. age = int("25")

C. age = bool("25")

D. age = "25" + 1

Multiple choice question

Answers

Correct answer: B

A. age = str("25")

B. age = int("25")

C. age = bool("25")

D. age = "25" + 1

Why:
In Python and OCR Exam Reference Language, int() casts a numeric string to an integer so it can be used in arithmetic or comparisons.

Exam Question

A ticket program receives the number of tickets as text before calculating the total cost.

Explain why the input must be cast and how an appropriate data type should be selected. (4 marks)

Exam Question

Answers

Answer guidance

- Input is initially received as a string. (1)
- int() converts the ticket quantity to an integer. (1)
- The integer can be used in arithmetic to calculate the cost. (1)
- A real data type may be used for a price containing a fractional part. (1)

Plenary – What have you learned?

In this lesson, you have learned that:

- **variables** store values that can change
- **constants** protect values that must stay fixed
- **identifiers** give stored values meaningful names
- **data types** determine valid values and operations
- **casting** converts a value so it can be used as another type

Exit ticket – Explain the choice

A school app stores a pupil's phone number and the number of tickets they have booked.

Explain why phoneNumber should be a string, while ticketQuantity should be an integer. (3 marks)

Exit ticket – Suggested answer

Answers

- **phoneNumber** should be a **string** because it is not used in arithmetic and may begin with 0.
- **ticketQuantity** should be an **integer** because it stores a whole number used in calculations.
- The **data type** is chosen from how the value will be used, not only how it looks.

Homework – Apply the learning

Design the data for a quiz game.

For each of these identifiers—playerName, score, averageScore, gameFinished and MAX_SCORE:

- choose the most suitable data type
- decide whether it is a variable or constant
- justify your decision in one sentence

Extension: Explain when the program might need casting.